

RAPPORT DE PROJET GRP 6

SynthCompanion

Jeu d'Escape Room Multijoueur

Desktop & Réalité Virtuelle (VR)

Module : Ingénierie et Maquette Numérique

Encadrant : Pr. HRIMECH Hamid

Année académique : 2025/2026

- AMARA YASSINE
- Serrar Zouhair
- Hajji Imrane
- Benouarrak Abdelbari
- Khachane Zakaria
- Hajar Oussama
- Naqi Boubker

Table des matières

1	Présentation du Projet	3
1.1	Introduction	3
1.2	Objectifs	4
1.3	Technologies utilisées	4
1.4	Répartition des tâches	5
2	Architecture du Projet	6
2.1	Structure des fichiers	6
2.2	Architecture réseau	7
2.3	Architecture du backend Python	7
3	Implémentation	9
3.1	Puzzle 1 — Keypad (Code numérique)	9
3.2	Puzzle 2 — Balle (Placement physique)	10
3.3	Puzzle 3 — Séquence (Cubes colorés)	12
3.4	Personnage IA — Alex	14
3.5	Système de logging comportemental	15
3.6	Analyse des données	16
4	Multijoueur et Réalité Virtuelle	18
4.1	Système multijoueur	18
4.1.1	NetworkManagerSetup	18
4.1.2	Tests avec ParrelSync	18
4.2	Intégration VR (Meta Quest 2)	19
4.2.1	Configuration XR	20
4.2.2	Contrôles VR	21
5	Guide d’Installation	22
5.1	Prérequis	22
5.2	Étapes d’installation	22
5.2.1	Cloner le projet	22
5.2.2	Installer les dépendances Python	22
5.2.3	Configurer FFmpeg	22
5.2.4	Configurer la clé API	22
5.2.5	Lancer le serveur backend	23
5.2.6	Ouvrir le projet Unity	23

6	Difficultés Rencontrées et Solutions	24
6.1	Problème 1 — Mouvement bloqué dans ParrelSync	24
6.2	Problème 2 — Double spawn du joueur	24
6.3	Problème 3 — Network Transform causait une translation	25
6.4	Problème 4 — XR Controller (Action-based) introuvable	25
7	Conclusion	26
7.1	Bilan du projet	26
7.2	Écran de fin de partie	26
7.3	Vidéo de démonstration	27

1. Présentation du Projet

1.1 Introduction

SynthCompanion est un jeu d'écape room coopératif en réseau, développé dans le cadre du module **Ingénierie et Maquette Numérique**. Le projet met en œuvre deux modes de jeu simultanés :

Mode Desktop : contrôle via clavier et souris

Mode VR : contrôle via casque Meta Quest 2

Deux joueurs collaborent en réseau pour résoudre 3 puzzles dans une salle mystérieuse, guidés par un personnage IA nommé **Alex**.

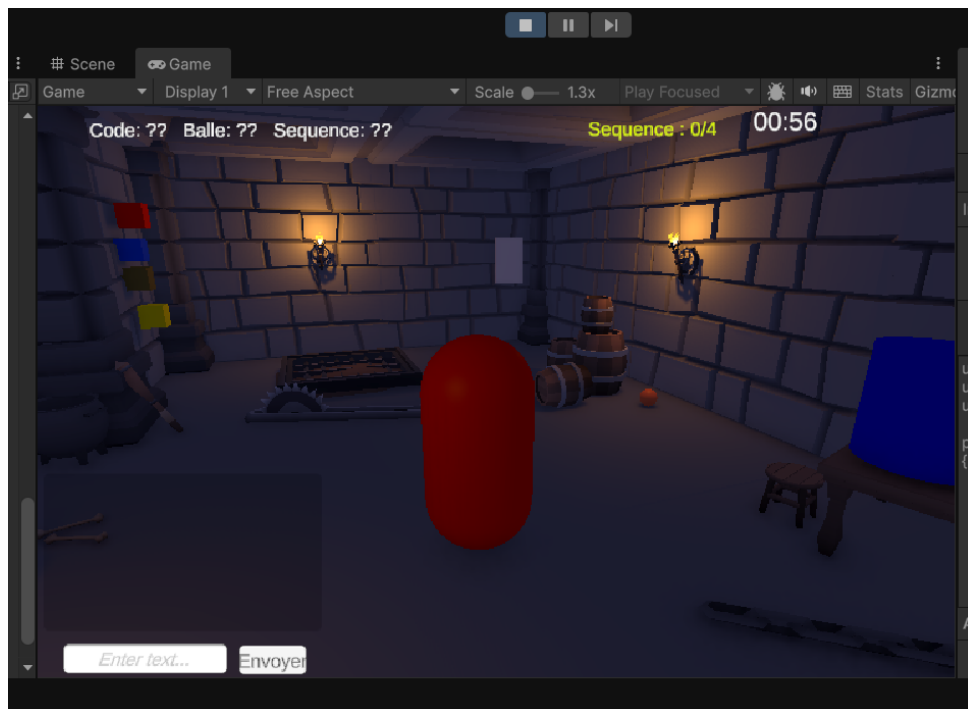


FIGURE 1.1 – Vue générale du jeu SynthCompanion

1.2 Objectifs

Objectifs du projet

1. Développer un jeu multijoueur en réseau avec Unity Netcode

2. Intégrer la réalité virtuelle (Meta Quest 2) avec XR Interaction Toolkit

3. Implémenter un personnage IA conversationnel (LLM Groq)

4. Permettre la reconnaissance et la synthèse vocale (Whisper + Edge TTS)

5. Assurer la compatibilité Desktop/VR dans la même session réseau

1.3 Technologies utilisées

Technologie	Utilisation
Unity 6.3 LTS	Moteur de jeu principal
XR Interaction Toolkit 3.3.1	Support VR, interactions contrôleurs
Meta Quest 2	Casque de réalité virtuelle
Python 3.11 + FastAPI	Serveur backend IA
Groq API (LLM)	Intelligence artificielle conversationnelle
OpenAI Whisper (STT)	Reconnaissance vocale
Edge TTS	Synthèse vocale
ParrelSync	Tests multijoueur en local
Git / GitHub	Gestion de versions

TABLE 1.1 – Technologies utilisées dans le projet

1.4 Répartition des tâches

Membre	Rôle	Contributions
AMARA YASSINE	Lead Dev & Réseau	Réseau Netcode, DesktopPlayer, NetworkManagerSetup, intégration VR, ParrelSync, XR Toolkit, ChatManager, Canvas UI
Khachane Zakaria	IA & Backend	Serveur FastAPI, Groq LLM, AI-Player, LLMConnector, Session-Logger, GameController
Serrar Zouhair	Gameplay & Physique	BallDragger, PlacementPuzzle, Target, XR Grab Interactable
Hajar Oussama	Audio & Voix	VoiceCapture, TTSPPlayer, Sav-Wav, micro Quest 2
Hajji Imrane	Puzzles	KeypadPuzzle, SequencePuzzle, PuzzleButton, PuzzleManager
Naqi Boubker	Scène 3D	GameRoom (murs, torches, éclairage), matériaux, préfabs
Benouarrak Abdelbari	QA & Documentation	Tests multijoueur, rapport de bugs, README, validation

TABLE 1.2 – Répartition des tâches de l'équipe

2. Architecture du Projet

2.1 Structure des fichiers

Listing 2.1 – Structure du projet Unity

```
1 SynthCompanion/  
2 |-- Assets/  
3 |   |-- Prefabs/  
4 |   |   |-- DesktopPlayer.prefab    # Joueur clavier/souris  
5 |   |   |-- VRPlayer.prefab         # Joueur Meta Quest 2  
6 |   |   |-- AIPlayer.prefab         # Personnage IA Alex  
7 |   |   |-- Ball.prefab             # Balle puzzle 2  
8 |   |-- Scenes/  
9 |   |   |-- GameRoom.unity          # Scene principale  
10 |   |-- Scripts/  
11 |       |-- Network/               # Scripts reseau  
12 |       |-- AI/  
13 |       |-- Puzzles/               # Scripts puzzles  
14 |       |-- Data/  
15 |       |-- VR/                     # Scripts voix/audio  
16 |-- backend/  
17 |   |-- main.py                     # Serveur FastAPI  
18 |   |-- Analyse.py  
19 |   |-- .env                         # Cle API (non versionne)
```

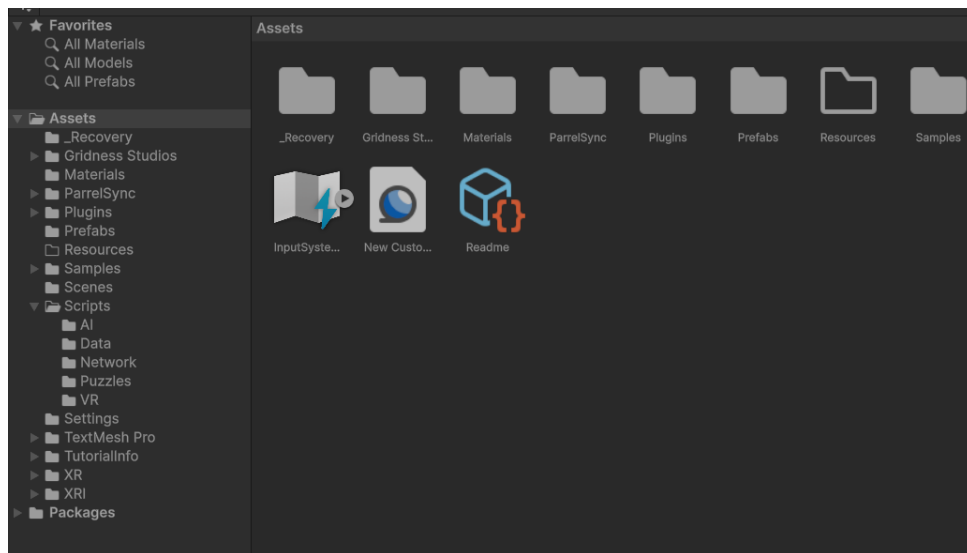


FIGURE 2.1 – Structure des dossiers du projet dans Unity

2.2 Architecture réseau

Le projet utilise **Unity Netcode for GameObjects** avec un modèle **Host/Client** :

Host : premier joueur, héberge la partie et joue simultanément

Client : deuxième joueur, se connecte au Host

ServerRpc : appels client → serveur (ex : appuyer un bouton puzzle)

ClientRpc : diffusion serveur → tous les clients (ex : mise à jour du chat)

Détection automatique Desktop/VR

La détection du mode de jeu est automatique via :

```
bool isVR = UnityEngine.XR.XRSettings.isDeviceActive;
GameObject prefab = isVR ? vrPlayerPrefab :
    desktopPlayerPrefab;
```

Si un casque VR est détecté et actif, VRPlayer spawne. Sinon, DesktopPlayer spawne.

2.3 Architecture du backend Python

Le backend est composé d'un serveur **FastAPI** qui expose trois endpoints principaux :

POST /dialogue : reçoit le contexte de jeu et l'historique du chat, retourne une réponse générée par Groq (Llama 3)

POST /tts : reçoit un texte, génère un fichier audio MP3 via Edge TTS

POST /stt : reçoit un chemin de fichier WAV, retourne la transcription via Whisper

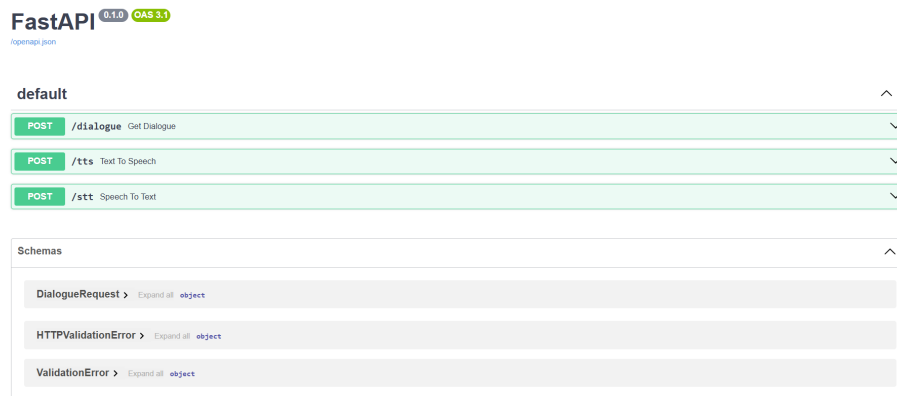


FIGURE 2.2 – Interface Swagger du backend FastAPI

3. Implémentation

3.1 Puzzle 1 — Keypad (Code numérique)

Le joueur doit trouver et entrer un code à 4 chiffres sur un clavier numérique.

Fonctionnement :

1. Le panel s'ouvre automatiquement par proximité (distance configurable)
2. Le joueur clique sur les boutons chiffres (0-9)
3. Le code entré est comparé au code secret via ServerRpc
4. Si correct : `puzzle1State.Value = 1` et le panel se ferme

Listing 3.1 – KeypadPuzzle.cs — Logique de vérification

```
1 public void TryCode()
2 {
3     TryCodeServerRpc(enteredCode);
4 }
5
6 [ServerRpc(RequireOwnership = false)]
7 void TryCodeServerRpc(string entered)
8 {
9     if (entered == secretCode)
10    {
11        PuzzleManager.Instance.puzzle1State.Value = 1;
12        HidePanelClientRpc();
13    }
14    else
15    {
16        ResetCodeClientRpc();
17    }
18 }
```



FIGURE 3.1 – Panel Keypad avec les boutons chiffres

3.2 Puzzle 2 — Balle (Placement physique)

Le joueur doit déplacer une balle jusqu'à une cible précise dans la scène.

Fonctionnement :

Desktop : clic et drag avec la souris via `BallDragger.cs`

VR : saisir la balle avec la gâchette via `XR Grab Interactable`

Détection de la cible via trigger collider

Synchronisation réseau de la position de la balle

Listing 3.2 – `BallDragger.cs` — Déplacement par clic et glisser

```
1 void Update()
2 {
3     if (cam == null) cam = Camera.main;
4
5     // Clic gauche souris utilise le nouveau Input
6     // System
7     bool mouseDown = Mouse.current.leftButton.
8         wasPressedThisFrame;
9     bool mouseUp = Mouse.current.leftButton.
10        wasReleasedThisFrame;
11     bool mouseHeld = Mouse.current.leftButton.isPressed;
12
13     Vector2 mousePos = Mouse.current.position.ReadValue();
14
15     if (mouseDown)
16     {
17         // V rifie si on clique sur la balle
18     }
19 }
```

```
15         Ray ray = cam.ScreenPointToRay(mousePos);
16         if (Physics.Raycast(ray, out RaycastHit hit, 20f))
17         {
18             if (hit.transform == transform)
19             {
20                 isDragging = true;
21                 if (rb != null)
22                 {
23                     rb.isKinematic = true;
24                     rb.linearVelocity = Vector3.zero;
25                 }
26             }
27         }
28     }
29
30     if (mouseUp)
31     {
32         isDragging = false;
33         if (rb != null)
34             rb.isKinematic = false;
35     }
36
37     if (isDragging && mouseHeld)
38     {
39         Ray ray = cam.ScreenPointToRay(mousePos);
40         Plane plane = new Plane(Vector3.up, new Vector3(0,
41             dragHeight, 0));
42         if (plane.Raycast(ray, out float dist))
43         {
44             Vector3 targetPos = ray.GetPoint(dist);
45             targetPos.y = dragHeight;
46             if (IsServer)
47                 transform.position = Vector3.Lerp(transform.
48                     position, targetPos, 0.3f);
49             else
50                 UpdatePositionServerRpc(targetPos);
51         }
52     }
53 }
```



FIGURE 3.2 – La balle et la cible dans la scène



FIGURE 3.3 – Déplacement de la balle vers la zone cible

3.3 Puzzle 3 — Séquence (Cubes colorés)

Le joueur doit appuyer sur 4 cubes colorés dans le bon ordre.

Ordre correct : Rouge (0) → Vert (2) → Bleu (1) → Jaune (3)

Listing 3.3 – SequencePuzzle.cs — Vérification de la séquence

```
1 [ServerRpc(RequireOwnership = false)]
2 public void PressButtonServerRpc(int buttonId)
3 {
4     playerOrder.Add(buttonId);
5     UpdateProgressClientRpc(playerOrder.Count, buttonId);
```

```
6
7     if (playerOrder.Count == 4)
8     {
9         bool correct = true;
10        for (int i = 0; i < 4; i++)
11            if (playerOrder[i] != correctOrder[i]) { correct
              = false; break; }
12
13        if (correct)
14        {
15            PuzzleManager.Instance.puzzle3Complete.Value =
              true;
16            ShowSuccessClientRpc();
17        }
18        else
19        {
20            playerOrder.Clear();
21            ShowErrorClientRpc();
22        }
23    }
24 }
```

Compatibilité Desktop/VR :

Desktop : cliquer **esc** pour libérer la souris + cliquer sur les cubes → `PressButtonServerRpc()`

VR : XR Simple Interactable + événement `Select Entered` → `Activate()`

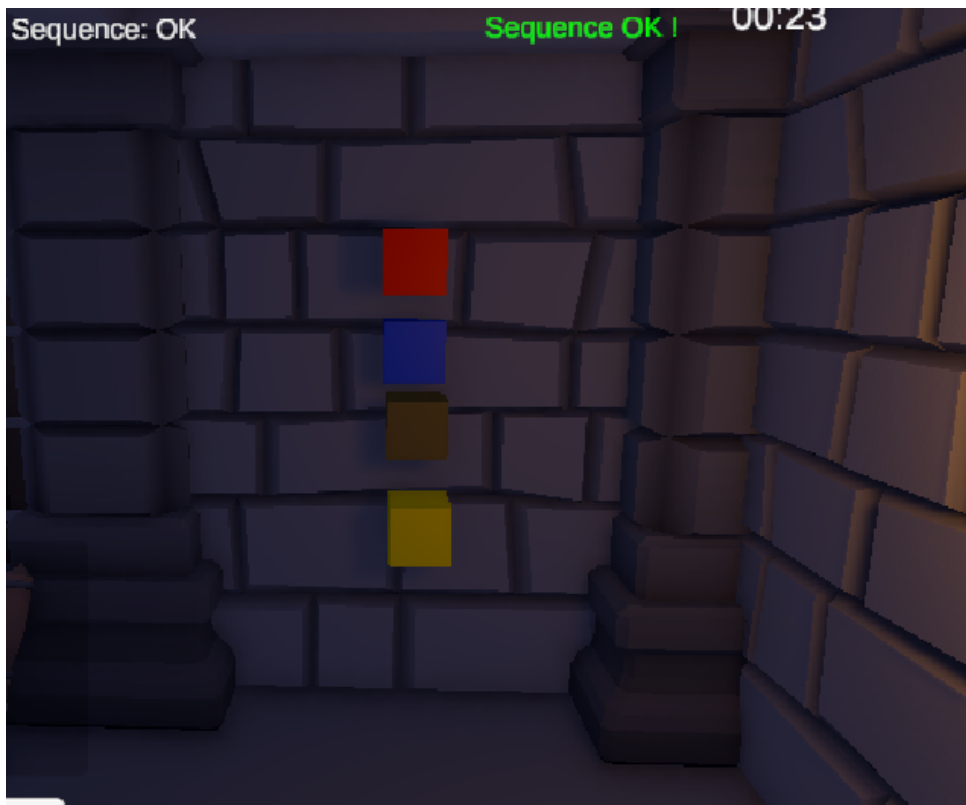


FIGURE 3.4 – Les 4 cubes colorés avec "Sequence OK!" après résolution



FIGURE 3.5 – Feedback visuel de progression de la séquence

3.4 Personnage IA — Alex

Le personnage Alex guide les joueurs via le chat avec une IA conversationnelle.

Pipeline IA

1. Joueur parle (touche T Desktop / bouton X VR)
2. `VoiceCapture.cs` enregistre le micro
3. Whisper (STT) transcrit l'audio en texte
4. Groq LLM génère une réponse contextuelle
5. Edge TTS synthétise la réponse en audio
6. `TTSPPlayer.cs` joue l'audio pour tous les clients
7. La réponse s'affiche dans le chat via `ChatManager`

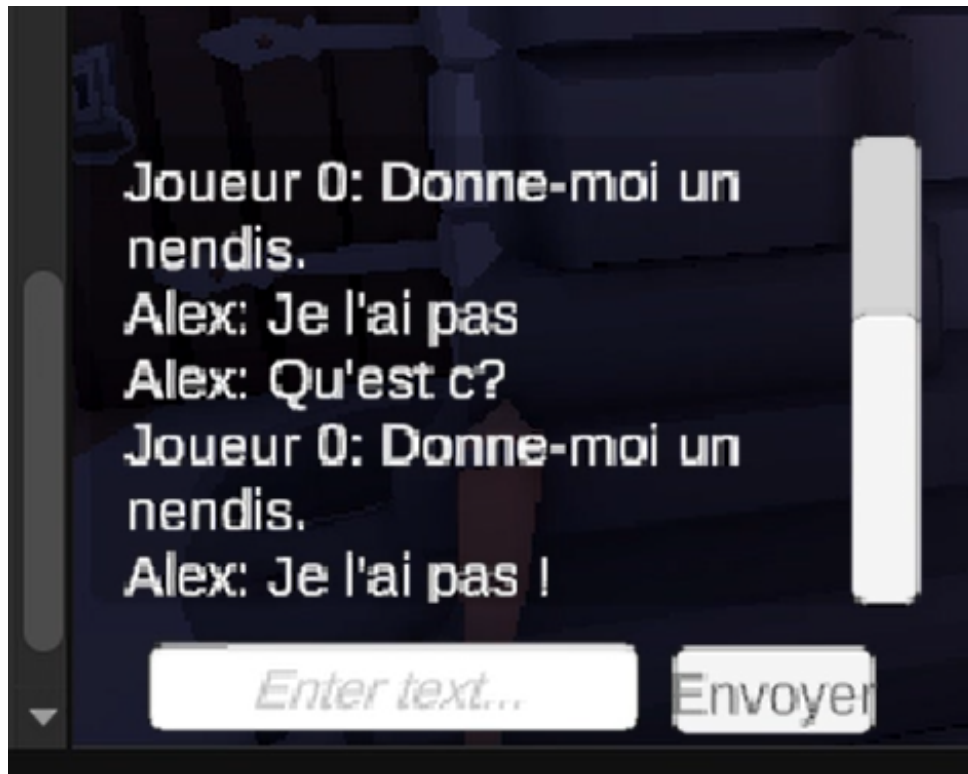


FIGURE 3.6 – Chat en temps réel avec le personnage IA Alex

3.5 Système de logging comportemental

Chaque action en jeu est enregistrée dans un fichier JSON pour l'analyse post-session.

Listing 3.4 – SessionLogger.cs — Enregistrement des actions

```

1 public void Log(string playerId, string action, string target =
   "")
2 {
3     var pm = PuzzleManager.Instance;
4     string state = pm != null
5         ? $"P1:{pm.puzzle1State.Value}□P2:{pm.puzzle2State.Value
6         }
7         : "unknown";
8     events.Add(new LogEvent
9     {
10         timestamp = DateTime.Now.ToString("HH:mm:ss.fff"),
11         playerId = playerId,
12         action = action,
13         target = target,
14         puzzleState = state
15     });
16 }

```

Les fichiers JSON sont sauvegardés dans :

```
C:\Users\NOM\AppData\LocalLow\DefaultCompany\SynthCompanion\
```



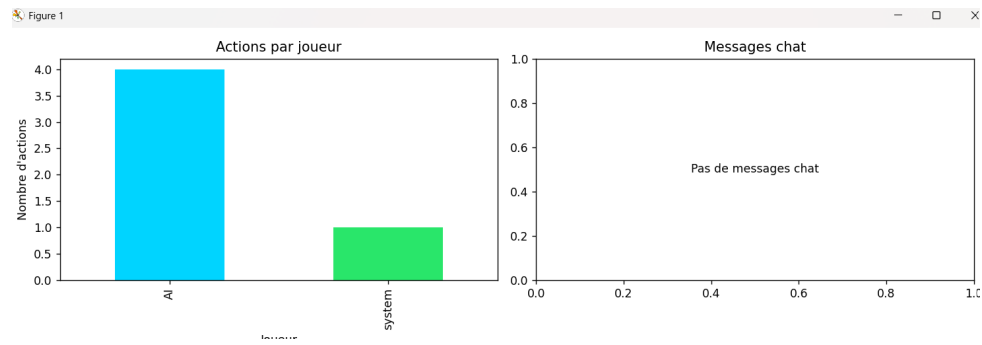
```
{ } session_db40c269.json X
C: > Users > MDT > AppData > LocalLow > DefaultCompany > SynthCompanion > { } s

1  {
2    "sessionId": "db40c269",
3    "startTime": "2026-05-31 21:30:54",
4    "totalEvents": 5,
5    "events": [
6      {
7        "timestamp": "21:31:46.653",
8        "playerId": "AI",
9        "action": "spoke",
10       "target": "j ai une idee",
11       "puzzleState": "P1:1 P2:1 P3:False"
12     },
13     {
14       "timestamp": "21:31:59.676",
15       "playerId": "AI",
16       "action": "spoke",
17       "target": "on manque de temps!!",
18       "puzzleState": "P1:1 P2:1 P3:False"
19     },
20     {
21       "timestamp": "21:32:25.217",
22       "playerId": "AI",
23       "action": "spoke",
24       "target": "j ai une idee",
25       "puzzleState": "P1:1 P2:1 P3:False"
26     },
27     {
28       "timestamp": "21:32:33.442",
29       "playerId": "system",
30       "action": "game_complete",
31       "target": "",
32       "puzzleState": "P1:1 P2:1 P3:True"
33     },
34     {
35       "timestamp": "21:32:34.220",
36       "target": "j ai une idee",
37       "puzzleState": "P1:1 P2:1 P3:False"
38     }
39   ]
40 }
```

FIGURE 3.7 – Exemple de fichier JSON d’une session de jeu

3.6 Analyse des données

Après chaque session, le script `analyse.py` génère des graphiques comportementaux.

FIGURE 3.8 – Analyse comportementale générée par `analyse.py`

4. Multijoueur et Réalité Virtuelle

4.1 Système multijoueur

4.1.1 NetworkManagerSetup

Le composant `NetworkManagerSetup` gère le démarrage de la partie et le spawn des joueurs :

Listing 4.1 – Spawn automatique Desktop ou VR

```
1 void OnClientConnected(ulong clientId)
2 {
3     if (!NetworkManager.Singleton.IsServer) return;
4     bool isVR = UnityEngine.XR.XRSettings.isDeviceActive;
5     GameObject prefab = isVR ? vrPlayerPrefab :
6         desktopPlayerPrefab;
7     if (prefab != null)
8     {
9         var player = Instantiate(prefab);
10        player.GetComponent<NetworkObject>()
11            .SpawnAsPlayerObject(clientId);
12    }
```

4.1.2 Tests avec ParrelSync

ParrelSync permet de tester le multijoueur avec deux instances d'Unity sur le même PC :

1. Menu Unity : ParrelSync → Clones Manager
2. Cliquer **Open in New Editor** sur le clone
3. Éditeur principal → **Play** → **Host**
4. Clone → **Play** → **Join**
5. Cliquer dans la fenêtre Game du clone pour donner le focus clavier

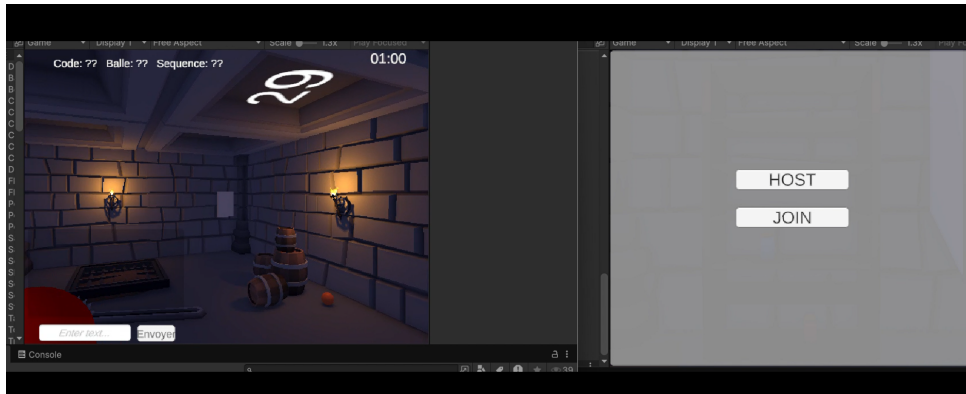


FIGURE 4.1 – Deux fenêtres Unity côte à côte (Host + Clone)



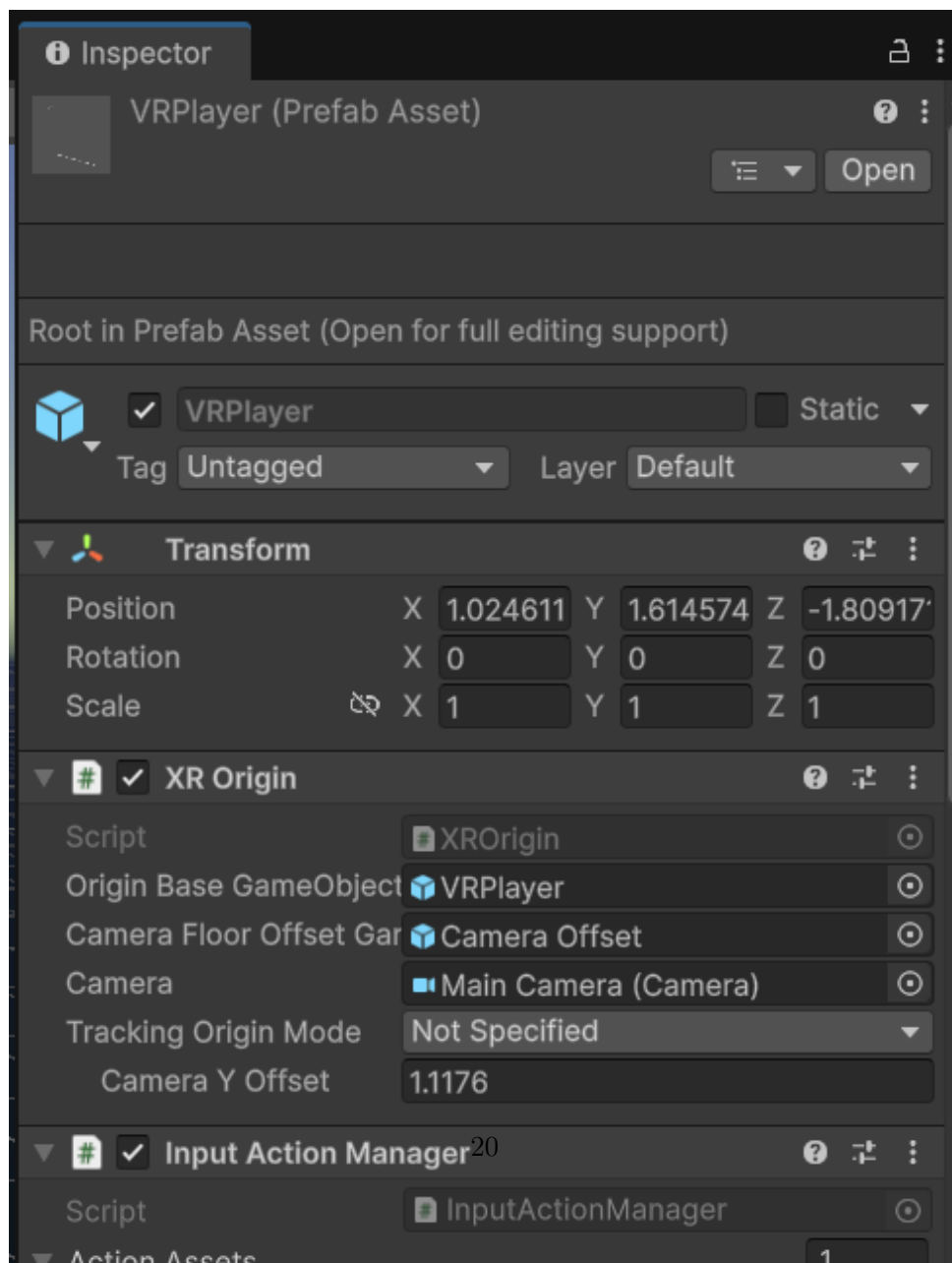
FIGURE 4.2 – Les deux joueurs et l'IA Alex dans la scène multijoueur

4.2 Intégration VR (Meta Quest 2)

Composant	Rôle
XR Plugin Management	Gestion du plugin Oculus
XR Interaction Toolkit 3.3.1	Interactions VR (grab, pointer, UI)
XR Ray Interactor	Rayon laser depuis les contrôleurs
XR Interactor Line Visual	Rendu visuel du rayon
Continuous Move Provider	Déplacement au joystick gauche
Continuous Turn Provider	Rotation au joystick droit
Input Action Manager	Gestion des inputs Quest 2
XR Grab Interactable	Saisir la balle avec la gâchette
XR Simple Interactable	Activer les cubes de séquence

TABLE 4.1 – Composants VR du prefab VRPlayer

4.2.1 Configuration XR



4.2.2 Contrôles VR

Contrôle	Action
Joystick gauche	Se déplacer
Joystick droit	Tourner
Gâchette (index)	Interagir / Saisir un objet
Bouton X	Activer la reconnaissance vocale
Tête	Regarder / orienter la caméra

TABLE 4.2 – Contrôles Meta Quest 2

5. Guide d'Installation

5.1 Prérequis

Unity 6.3 LTS → <https://unity.com>

Python 3.11 → <https://www.python.org/downloads/>

FFmpeg → <https://www.gyan.dev/ffmpeg/builds/>

Clé API Groq (gratuite) → <https://console.groq.com>

Meta Quest 2 (optionnel pour le mode VR)

5.2 Étapes d'installation

5.2.1 Cloner le projet

```
git clone https://github.com/Yassine-Amara/synth-companion.git
```

5.2.2 Installer les dépendances Python

```
pip install fastapi uvicorn groq edge-tts openai-whisper python-dotenv
```

5.2.3 Configurer FFmpeg

1. Télécharger FFmpeg depuis <https://www.gyan.dev/ffmpeg/builds/>
2. Extraire et placer dans C:\ffmpeg
3. Ajouter C:\ffmpeg\bin dans les variables d'environnement PATH
4. Vérifier avec `ffmpeg -version`

5.2.4 Configurer la clé API

Créer le fichier backend/.env :

```
GROQ_API_KEY=gsk_VOTRE_CLE_ICI
```

Important

Ne jamais partager cette clé. Ne jamais la pousser sur GitHub. Ajouter `.env` dans le fichier `.gitignore`.

5.2.5 Lancer le serveur backend

```
cd backend
python main.py
```

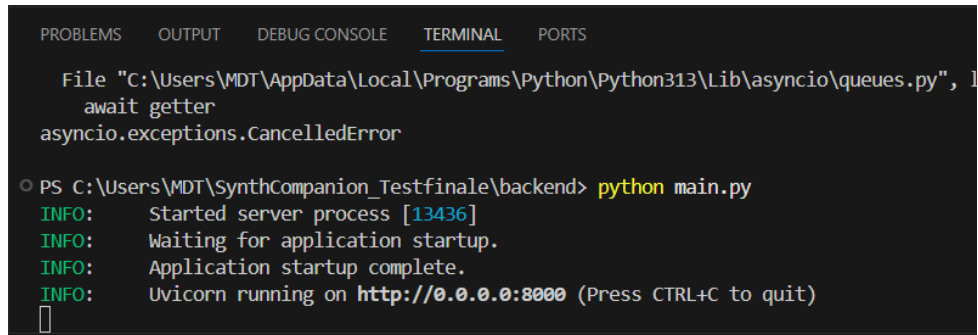


FIGURE 5.1 – Backend Python lancé avec succès

5.2.6 Ouvrir le projet Unity

1. Ouvrir Unity Hub → **Open** → sélectionner le dossier cloné
2. Attendre le chargement des packages
3. Ouvrir la scène : **Assets / Scenes / GameRoom**
4. Appuyer **Play** → **Host**

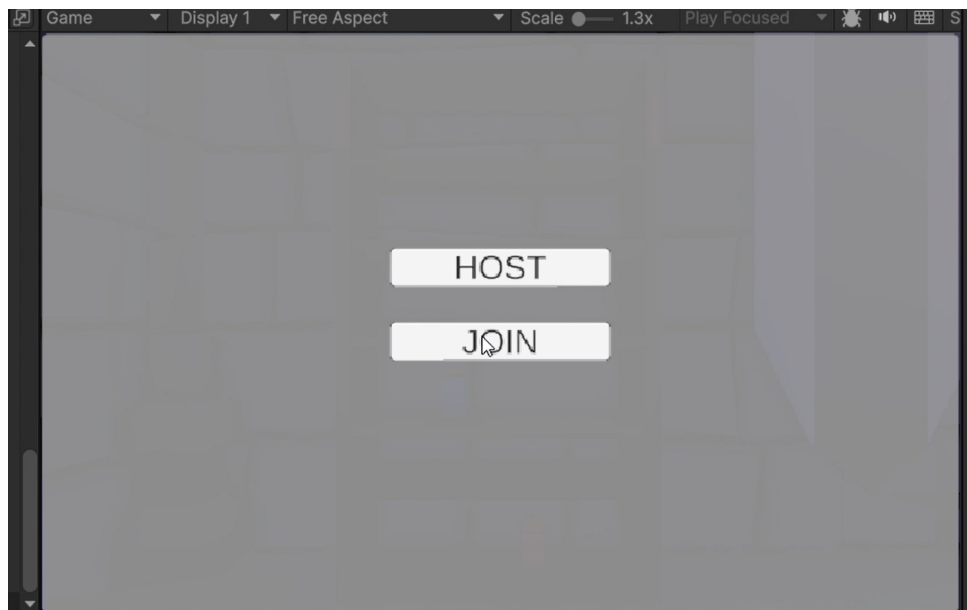


FIGURE 5.2 – Écran de lobby avec les boutons HOST et JOIN

6. Difficultés Rencontrées et Solutions

6.1 Problème 1 — Mouvement bloqué dans ParrelSync

Problème

Dans le clone ParrelSync, après avoir cliqué **Join**, le joueur ne pouvait pas se déplacer. Seule la caméra pouvait bouger verticalement.

Cause : `cc = GetComponent<CharacterController>()` était dans `Start()`, qui s'exécute avant que `IsOwner` soit établi dans le clone. La souris n'était jamais lockée et les inputs ne passaient pas.

Solution : Déplacer l'initialisation dans `OnNetworkSpawn()` après la vérification `IsOwner`.

Listing 6.1 – Correction dans DesktopPlayer.cs

```
1 public override void OnNetworkSpawn()  
2 {  
3     if (!IsOwner) return;  
4     cc = GetComponent<CharacterController>(); // D plac ici  
5     velocity = Vector3.zero;  
6     // ... reste de l'initialisation  
7 }
```

6.2 Problème 2 — Double spawn du joueur

Problème

Après avoir cliqué **Host**, deux capsules apparaissaient dans la scène. La caméra suivait la mauvaise.

Cause : Le joueur était spawnné deux fois — une fois par le **Player Prefab** assigné dans le `NetworkManager`, et une autre fois par notre code dans `NetworkManagerSetup`.

Solution : Vider le champ **Player Prefab** dans le composant `NetworkManager`, et utiliser `OnClientConnectedCallback` pour spawner une seule fois par connexion.

6.3 Problème 3 — Network Transform causait une translation

Problème

Le joueur glissait tout seul après le spawn malgré le fix du CharacterController.

Cause : Le composant **Network Transform** écrasait la position locale avec la position synchronisée du serveur, créant un conflit avec le mouvement local.

Solution : Supprimer le composant Network Transform du prefab DesktopPlayer.

6.4 Problème 4 — XR Controller (Action-based) introuvable

Problème

Impossible de trouver XR Controller (Action-based) avec XR Interaction Toolkit 3.3.1.

Cause : Ce composant a été supprimé dans la version 3.x du XR Interaction Toolkit.

Solution : Utiliser **XR Ray Interactor** directement sur les contrôleurs, avec **Continuous Move Provider**, **Continuous Turn Provider**, et le fichier **XRI Default Input Actions** pour les bindings.

7. Conclusion

7.1 Bilan du projet

Le projet SynthCompanion a permis de mettre en pratique les concepts suivants :

Résultats obtenus

- Jeu multijoueur fonctionnel en réseau (Host/Client)
- Compatibilité Desktop (clavier/souris) et VR (Meta Quest 2)
- 3 puzzles interactifs synchronisés sur le réseau
- Personnage IA conversationnel avec voix (Alex)
- Reconnaissance vocale (Whisper) et synthèse vocale (Edge TTS)
- Tests multijoueur validés avec ParrelSync
- Système de logging comportemental en JSON
- Analyse des données avec Python/pandas
- Code versionné sur GitHub avec README détaillé

7.2 Écran de fin de partie

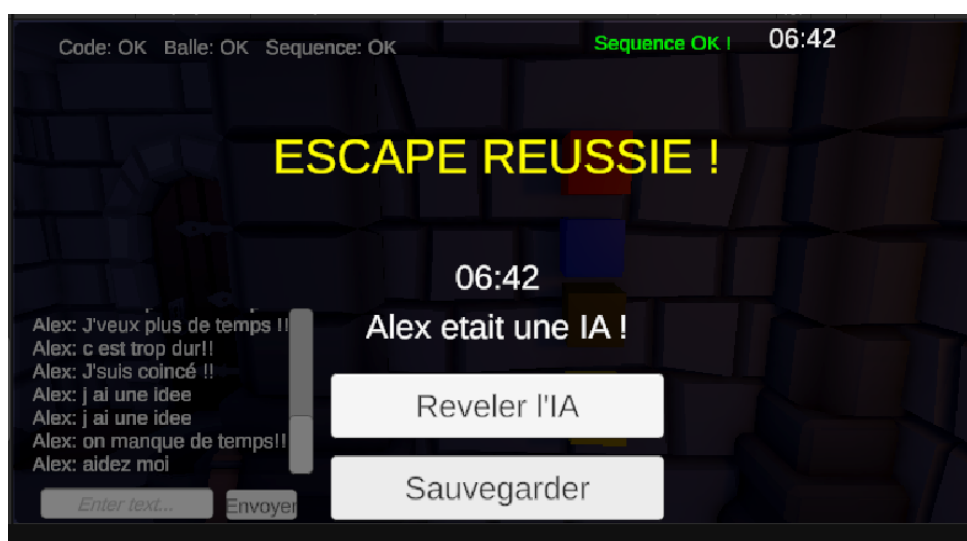


FIGURE 7.1 – Écran de fin de partie avec révélation de l'IA

7.3 Vidéo de démonstration

Une vidéo de démonstration complète du jeu est disponible à l'adresse suivante :

<https://drive.google.com/file/d/1x3s8utLy-9jHhkZGjEU-k30dqh4a1Ktn/view?usp=sharing>